

sllr Music Observatory

7 notes · Siirsuite

1. From recordings to a frequency map
2. What the sound descriptors measure
3. Pitch classes and timbre coordinates
4. Statistics and recurring structure
5. From spectral change to a beat pulse
6. How sound moves the ink
7. Growing music from simple rules

From recordings to a frequency map

The local analysis pipeline, sampling, and the time–frequency trade-off.

01.01 · Three ways to explore sound

EXPLANATION

sllr is a Flutter music observatory with three activities: analyze recordings or microphone input, visualize sound as moving ink, and generate music with cellular automata. Audio and analysis stay on the device.

The analyzer follows Signal → Descriptors → Statistics → Structure → Inspector → Experiments. A shared playback cursor connects the waveform, spectral view, and measurements. The inspector explains the selected frame and its units; the gain experiment checks how measurements respond to a controlled change.

These lessons describe the current Flutter project. They complement the broader Computational Musicology collection with the actual conventions in sllr.

MATHS

A recording is a discrete sequence $x[n]$ sampled at f_s samples per second:

$$t_n = \frac{n}{f_s}, \quad T = \frac{L}{f_s}.$$

Here L is the number of samples. Imported channels are averaged into mono before analysis. For C channels,

$$x[n] = \frac{1}{C} \sum_{c=1}^C x_c[n].$$

Amplitude is relative to digital full scale, not calibrated sound pressure. Opposite-phase stereo channels can cancel during averaging.

01.02 · Frames and the Hann window

EXPLANATION

The offline analyzer resamples to 22,050 Hz and processes overlapping frames. The default window contains 1,024 samples and the requested hop is 512 samples. Supported window sizes are 512, 1,024, and 2,048.

A Hann window reduces abrupt frame-edge discontinuities before the Fourier transform. Larger windows separate nearby frequencies more clearly but smear fast changes over a longer interval.

To cap long recordings at about 6,000 descriptor frames, the analyzer can increase the hop. Use the effective settings recorded with an analysis rather than assuming every run has the default time resolution.

MATHS

For frame m , window size N , and hop H ,

$$w[n] = \frac{1}{2} - \frac{1}{2} \cos\left(\frac{2\pi n}{N-1}\right),$$
$$X_m[k] = \sum_{n=0}^{N-1} x[mH + n] w[n] e^{-2\pi i k n / N}.$$

At $f_s = 22050$, $N = 1024$ gives a bin spacing $f_s/N \approx 21.53$ Hz and a window duration $N/f_s \approx 46.44$ ms. A hop of 512 gives about 23.22 ms between frame starts.

01.03 · Reading a spectrogram honestly

EXPLANATION

The spectrogram displays one-sided FFT magnitudes, including DC and Nyquist. Its colors show the logarithm of the frame-transform magnitude, not a calibrated power spectral density.

The offline frame timestamp refers to its center. Short inputs are zero-padded for a first frame, and timestamps are bounded by the recording duration. The spectrum is the frame nearest the cursor; it does not directly identify a fundamental pitch.

Live beat tracking uses trailing frames with timestamps at the end of the audio window instead. Do not confuse these timing conventions when comparing live beats with offline descriptor plots.

MATHS

Write $A_m[k] = |X_m[k]|$ and $P_m[k] = A_m[k]^2$. Frequency and nominal offline frame time are

$$f_k = \frac{k f_s}{N}, \quad t_m = \frac{mH + N/2}{f_s}.$$

Spectrogram color uses

$$D_m[k] = 20 \log_{10}(\max(A_m[k], 10^{-12})) .$$

The floor keeps silence finite. Unnormalized FFT magnitude depends on the window and transform size, so numerical color values should not be compared as calibrated sound levels.

What the sound descriptors measure

Level, spectral shape, and change—with the exact weighting and silence conventions.

02.01 · Level and spectral balance

EXPLANATION

RMS measures amplitude without cancellation between positive and negative samples. sllr computes it on the unwindowed frame. It is neither sound-pressure level nor a complete model of perceived loudness.

Centroid measures the spectrum's balance point and spread measures its width around that point. Both use magnitude weights, including the DC bin. A bright sound often has a high centroid, but the interpretation depends on the recording and listening context.

A useful experiment is to multiply the same floating-point recording by a nonzero gain: RMS should scale while centroid and spread should stay constant, away from silence thresholds and numerical floors.

MATHS

With raw frame samples $x_m[n]$ and magnitude weights $a_m[k] = A_m[k] / \sum_j A_m[j]$,

$$\text{RMS}_m = \sqrt{\frac{1}{N} \sum_n x_m[n]^2},$$

$$c_m = \sum_k f_k a_m[k], \quad s_m = \sqrt{\sum_k (f_k - c_m)^2 a_m[k]}.$$

For $y[n] = gx[n]$ with $g \neq 0$,

$$\text{RMS}(y) = |g| \text{RMS}(x), \quad c(y) = c(x).$$

This prediction assumes no clipping. The in-app experiment operates in floating point without playback clipping.

02.02 · Distribution of spectral power

EXPLANATION

Entropy describes how evenly power is distributed across bins. Flatness compares the geometric and arithmetic means of power. Roll-off finds the first frequency bin containing 85% of cumulative power.

These descriptors answer different questions. Entropy is not a musical-complexity score, flatness is not a categorical noise detector, and roll-off is a chosen descriptive threshold.

The implementation returns zero for silence-derived spectral descriptors. Those zeros are placeholders, not evidence that silence has a meaningful brightness or spectral distribution.

MATHS

Let $K = N/2 + 1$ and $p_k = P[k] / \sum_j P[j]$. Normalized entropy is

$$E = -\frac{\sum_k p_k \ln p_k}{\ln K}, \quad 0 \ln 0 = 0.$$

With $\epsilon = 10^{-24}$, flatness is

$$F = \frac{\exp(K^{-1} \sum_k \ln \max(P[k], \epsilon))}{K^{-1} \sum_k P[k]}.$$

The result is clamped to $[0, 1]$. Roll-off is

$$r = f_{k_*}, \quad k_* = \min \left\{ k : \sum_{j=0}^k P[j] \geq 0.85 \sum_j P[j] \right\}.$$

The code treats $\sum_k P[k] < 10^{-24}$ as silent. Floors matter for very quiet signals.

02.03 · Changes and crossings

EXPLANATION

Spectral flux compares successive normalized magnitude spectra and keeps only positive changes. It often responds to attacks, but silence-to-sound transitions also create peaks. The first frame has zero flux because it has no predecessor.

Zero-crossing rate counts changes between the nonnegative and negative sides of the waveform. It is sensitive to DC offset and noise; it is not a reliable pitch detector by itself.

Flux is normalized spectral change, so its units differ from RMS. Always read the units before comparing descriptor plots.

MATHS

For normalized magnitudes $a_m[k]$,

$$\text{Flux}_m = \sqrt{\sum_k \max(0, a_m[k] - a_{m-1}[k])^2}.$$

A spectrum whose magnitude sum is below the normalization threshold uses zero weights. Zero-crossing rate on the raw frame is

$$Z_m = \frac{1}{N-1} \sum_{n=1}^{N-1} |b_n - b_{n-1}|,$$

Here $b_n = \mathbf{1}[x_m[n] \geq 0]$. Zero belongs to the nonnegative side. The result is crossings per sample rather than crossings per second.

Pitch classes and timbre coordinates

FFT chroma, native ECQT, and the five MFCC coefficients.

03.01 · Folding octaves into chroma

EXPLANATION

Chroma combines energy from octave-related notes into twelve pitch classes. In sllr's ordinary descriptor pipeline, FFT bins between 27.5 and 4,186 Hz are assigned to the nearest equal-tempered note, then their magnitudes are summed and normalized.

This is a compact harmonic fingerprint, not a chord transcription. Harmonics, detuning, and coarse FFT spacing at low frequencies can distribute energy among several classes.

The structure view uses this FFT-derived chroma. The separate native CQT view has different windows and uses power when folding its pitch bins.

MATHS

For eligible positive-frequency bins,

$$q_k = \text{round}\left(69 + 12 \log_2 \frac{f_k}{440}\right).$$

The FFT chroma accumulator and normalization are

$$\tilde{C}[c] = \sum_{k: q_k \bmod 12 = c} A[k], \quad C[c] = \frac{\tilde{C}[c]}{\sum_d \tilde{C}[d]}.$$

A4 is 440 Hz and class zero is C. If the accumulator has negligible total magnitude, the code leaves a zero vector.

03.02 · Native constant-Q analysis

EXPLANATION

ECQT uses geometrically spaced frequencies and pitch-dependent window lengths. Low notes receive longer windows; high notes receive shorter ones. sllr's native wrapper exposes 84 pitch-bin magnitudes starting at C2.

Kernels are centered on the playback cursor, and the input is zero-padded at recording edges. Pitch analysis updates at approximately 10 Hz and reuses results between updates.

Long bass windows improve frequency discrimination at the cost of temporal precision. CQT pitch-class energy can be activated by harmonics; it is not a monophonic fundamental-frequency estimate.

MATHS

At twelve bins per octave, the nominal constant-Q parameters are

$$Q = \frac{1}{2^{1/12} - 1}, \quad f_k = 65.406391 \cdot 2^{k/12},$$

$$N_k = \left\lceil \frac{Q f_s}{f_k} \right\rceil, \quad f_s = 22050.$$

A conceptual pitch kernel measures a windowed complex sinusoidal projection at f_k ; ECQT evaluates sparse FFT-domain kernels. The native configuration uses Hamming windows and sparsification threshold 0.001. Folding magnitudes $|C_k|$ into pitch-class power gives

$$P[c] = \frac{\sum_{k \bmod 12=c} |C_k|^2}{\sum_k |C_k|^2}.$$

Silence yields zero instead of division by zero.

03.03 · MFCCs summarize the spectral envelope

EXPLANATION

MFCCs compress the broad shape of the spectrum through a mel-spaced filterbank, logarithms, and a cosine transform. slr uses 26 triangular filters and retains five coefficients, including coefficient zero.

These coefficients depend on the exact analysis conventions. The implementation has no pre-emphasis or liftering, and uses an unnormalized DCT-II. Coefficient zero includes level information.

Silence produces values determined by the logarithmic floor. Those values should not be interpreted as a measured timbre.

MATHS

Frequency is mapped to mel by

$$\text{mel}(f) = 2595 \log_{10}(1 + f/700).$$

For filter $B_b[k]$, define

$$E_b = \sum_k P[k] B_b[k], \quad \ell_b = \ln \max(E_b, 10^{-24}).$$

The retained coefficients are

$$c_j = \sum_{b=0}^{25} \ell_b \cos\left(\frac{\pi j(b + 1/2)}{26}\right), \quad j = 0, \dots, 4.$$

They are spectral-envelope coordinates rather than physical units or genre labels.

Statistics and recurring structure

Correlation, PCA, self-similarity, and candidate boundaries.

04.01 · Measurements that move together

EXPLANATION

The statistics view compares eight descriptors: RMS, centroid, spread, entropy, flatness, roll-off, flux, and zero-crossing rate. Standardization prevents the larger numerical scale of frequency-valued measurements from dominating PCA.

Correlation asks which descriptors vary together. Constant columns become zeros, including their correlation-matrix diagonal. Overlapping frames are dependent observations, so a large correlation does not establish causation or statistical significance.

Interval summaries use frame centers within the selected range. Averages and standard deviations summarize measurements; they do not replace listening or the distribution shown by a histogram.

MATHS

For M frames and descriptor j ,

$$\mu_j = \frac{1}{M} \sum_m x_{mj}, \quad \sigma_j = \sqrt{\frac{\sum_m (x_{mj} - \mu_j)^2}{M - 1}}.$$

When $M \geq 2$ and $\sigma_j \geq 10^{-12}$,

$$Z_{mj} = \frac{x_{mj} - \mu_j}{\sigma_j}, \quad R = \frac{Z^T Z}{M - 1}.$$

Constant columns use $Z_{mj} = 0$. With fewer than two frames, correlations are set to zero.

04.02 · A two-axis map of variation

EXPLANATION

PCA rotates the standardized descriptor cloud into directions of maximal variation. sllr displays the leading two component scores together with their loadings and explained fractions.

An axis is a mixture of measurements, not an automatically discovered musical label. Inspect the loadings to see what drives a direction. Eigenvector signs are arbitrary: a mirrored axis can represent the same PCA result.

This map is exploratory. It does not by itself classify tracks, identify causes, or validate clusters.

MATHS

The symmetric correlation matrix is diagonalized using a Jacobi eigensolver:

$$Rv_c = \lambda_c v_c, \quad \lambda_1 \geq \lambda_2 \geq \dots$$

Component scores and explained fractions are

$$s_{mc} = Z_m v_c, \quad e_c = \frac{\max(0, \lambda_c)}{\sum_j \max(0, \lambda_j)}.$$

The code uses zero fractions when the denominator is negligible. Only the first two components are displayed; their fractions need not sum to one.

04.03 · Similarity and novelty across time

EXPLANATION

The structure view compares FFT chroma vectors with cosine similarity. Up to 160 uniformly sampled frames bound the matrix's memory cost. Bright off-diagonal areas can indicate repeated pitch-class material.

A novelty curve compares neighboring blocks: internally similar regions that disagree across their border produce a positive value. Local peaks above the mean plus one standard deviation suggest boundaries, separated by at least one second.

These are candidates for inspection and manual annotation. Shared chroma need not mean the same musical section, and downsampling limits boundary precision.

MATHS

For chroma vectors C_i, C_j ,

$$S_{ij} = \frac{C_i \cdot C_j}{\|C_i\| \|C_j\|}.$$

Near-zero norm products return zero. For a four-frame neighborhood, let B be the preceding indices and A the following indices:

$$\nu(t) = \max(0, \bar{S}_{BB} + \bar{S}_{AA} - 2\bar{S}_{BA}).$$

Each block mean averages sixteen entries. A candidate satisfies $\nu(t) > \mu_\nu + \sigma_\nu$, a strict rise from its predecessor, and a non-strict fall to its successor. Endpoint neighborhoods are incomplete and remain zero.

From spectral change to a beat pulse

Tempo candidates and the causal PLP tracker implemented in sllr.

05.01 · Periodicity suggests tempo

EXPLANATION

The offline tempo plot autocorrelates the spectral-flux sequence. Repeated attacks reinforce one another at particular delays; converting those delays to beats per minute gives candidate tempos.

The search covers 40–240 BPM. Peaks can reflect subdivisions or every-other-beat patterns, so half- and double-tempo interpretations remain possible. A short or nonperiodic recording may have no useful tempo.

Autocorrelation scores describe repetition. They are not probabilities that a particular tempo is correct.

MATHS

For nonnegative flux Δ_m and lag ℓ ,

$$R[\ell] = \frac{\sum_{m=0}^{M-\ell-1} \Delta_m \Delta_{m+\ell}}{\sum_{m=0}^{M-1} \Delta_m^2}.$$

The conversion from lag to tempo is

$$\text{BPM}(\ell) = \frac{60f_s}{H\ell}.$$

The denominator uses total activation energy rather than a lag-dependent normalization. Negligible energy yields no tempo candidates.

05.02 · Past onsets vote for a pulse

EXPLANATION

The causal PLP tracker evaluates rhythmic periodicities from recent activation history. It chooses a tempo and phase, constructs a signed rhythmic kernel, and overlap-adds successive kernels into a pulse buffer.

slr uses spectral-flux activation rather than a trained onset model. The default tempo grid is 60–180 BPM in increments of two; its kernel setting is six seconds. These are distinct from the offline autocorrelation search.

The live audio front end uses trailing windows. Future portions of a pulse kernel are predictions from past information, not measurements of future audio.

MATHS

Let δ be the activation-frame period and $h = \max(2, \text{round}(T_k/(2\delta)))$. For lag $j = 0, \dots, h$,

$$w_j = \frac{1}{2} + \frac{1}{2} \cos(\pi j/h).$$

For tempo b on the grid, the implementation computes

$$z_b = \sum_{j=0}^h \Delta_{n-j} w_j e^{i2\pi(b/60)\delta j}.$$

It chooses $b_* = \arg \max_b |z_b|^2$ and $\phi = \arg z_{b_*}$, then adds the centered kernel

$$\kappa_n[j] = \frac{w_{|j|}}{h} \cos\left(2\pi \frac{b_*}{60} \delta j + \phi\right), \quad -h \leq j \leq h.$$

The pulse buffer shifts one step before each new contribution.

05.03 · Beat decisions and timing limits

EXPLANATION

A beat event requires a local pulse maximum, sufficient clipped pulse strength, non-negligible activation, a warm-up period, and enough time since the previous event. The default lookahead is zero and the threshold is 0.08.

The quantity named stability in the implementation is a clipped pulse sample, not a calibrated confidence probability. Increasing lookahead samples a predicted future pulse position.

Microphone capture, frame buffering, and display all add latency. The implementation does not establish zero device latency or the accuracy of a trained system. Weak onsets and tempo ambiguity remain practical limits.

MATHS

For lookahead L_a , the decision index is

$$d = \text{clamp}(\text{round}(h + L_a/\delta), 1, 2h - 1).$$

If Γ is the overlap-added pulse, stability is $s = \text{clamp}(\Gamma[d], 0, 1)$. A local maximum must satisfy

$$\Gamma[d] > \Gamma[d - 1], \quad \Gamma[d] \geq \Gamma[d + 1].$$

The elapsed-time conditions are

$$t \geq \min(T_k/2, 2), \quad t - t_{\text{last}} > \frac{24}{\max(1, b_*)}.$$

The refractory duration is 40% of the selected beat period. These conditions describe the current code, not a universal beat-tracking definition.

How sound moves the ink

Advection, pressure projection, and audio-driven dye injection.

06.01 · Dye is a tracer of motion

EXPLANATION

The visualizer pours colored dye into a moving velocity field. Audio energy drives injection; low and high frequency-band energy influence the forces. Touch supplies additional motion and dye.

The equations motivate an artistic incompressible-flow approximation. The simulation is not a water-surface model, and the visual patterns do not classify beats or genres.

Velocity and dye are separate fields. The visualizer uses a worker for motion and GPU-assisted dye rendering; the CPU fluid implementation makes the essential numerical operations explicit.

MATHS

Write velocity as $\mathbf{u}$, pressure as p , forcing as $\mathbf{f}$, and a dye channel as d . The model is

$$\begin{aligned}\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} &= -\nabla p + \mathbf{f}, & \nabla \cdot \mathbf{u} &= 0, \\ \partial_t d + \mathbf{u} \cdot \nabla d &= -\lambda d + s.\end{aligned}$$

Here s injects dye and λ controls fading. These are model equations; the app uses bounded grids and approximate numerical steps rather than an exact solution.

06.02 · Trace backward, then project

EXPLANATION

Semi-Lagrangian advection asks where a grid cell's contents came from. It traces backward through velocity and interpolates the previous field. This stays usable at interactive step sizes, although interpolation introduces numerical smoothing.

Pressure projection removes part of the velocity divergence. The CPU solver uses twelve pressure iterations and enforces zero normal velocity at the outer boundaries.

Finite iteration count, grid resolution, and boundary treatment all affect the result. A smooth-looking animation is not proof of an accurately resolved physical flow.

MATHS

For a field q , a backward trace gives

$$q^{\text{adv}}(\mathbf{x}) \approx q^n(\mathbf{x} - \Delta t \mathbf{u}^n(\mathbf{x})).$$

Bilinear interpolation samples off-grid locations. Projection is conceptually

$$\nabla^2 p = \nabla \cdot \mathbf{u}^*, \quad \mathbf{u}^{n+1} = \mathbf{u}^* - \nabla p.$$

The CPU code uses grid-scaled centered differences and a twelve-sweep pressure relaxation. It clamps Δt to $[0, 0.04]$ seconds and applies velocity drag $e^{-1.1\Delta t}$.

06.03 · Energy, persistence, and presentation

EXPLANATION

Sensitivity controls how strongly the audio injects motion and color; persistence controls how long the dye remains visible. Audio controls are eased rather than applied as abrupt jumps.

The CPU solver multiplies dye by a decay factor per simulation step. Its default decay is therefore sensitive to step frequency. Brightness controls presentation and does not change the audio measurement.

Distinguish a change in the sound from a change in the rendering settings before interpreting a more intense visual pattern.

MATHS

A simple energy-control mapping shown in the app's maths panel is

$$E = \text{clamp}(g \text{ RMS}, 0, 1).$$

For per-step decay ρ , dye evolves approximately as

$$d^{n+1} = \rho d^{\text{adv}}.$$

At fixed step duration, its equivalent continuous rate is $\lambda = -\ln(\rho)/\Delta t$. The CPU pixel mapping compresses nonnegative dye concentration with

$$I = 255(1 - e^{-d}).$$

This maps concentration to display intensity; it is not a loudness scale.

Growing music from simple rules

Elementary cellular automata, scale mapping, and WAV synthesis.

07.01 · A row of cells becomes a score

EXPLANATION

The generator evolves a row of binary cells using an elementary cellular-automaton rule. Each next cell depends on its left neighbor, itself, and its right neighbor. The row wraps around, so its ends are neighbors.

A rule number selects an output for each of the eight possible neighborhoods. The same seed and settings produce the same score; complex-looking patterns do not require random choices.

Each generation is an eighth-note step. Changing BPM changes the time between generations rather than the cellular rule itself.

MATHS

For binary cell a_i^t in a row of length W , define the neighborhood code

$$u_i^t = 4a_{i-1}^t + 2a_i^t + a_{i+1}^t,$$

with indices taken modulo W . For rule $r \in \{0, \dots, 255\}$,

$$a_i^{t+1} = \left(\left\lfloor \frac{r}{2^{u_i^t}} \right\rfloor \bmod 2 \right).$$

At tempo $b > 0$, the step duration is $\Delta t = 30/b$ seconds. The default generator uses rule 30, 64 steps, and 110 BPM.

07.02 · Newly active cells trigger scale notes

EXPLANATION

Only newly activated cells trigger notes. A cell that stays active does not retrigger. On the first step, every active seed cell counts as newly activated.

Pairs of adjacent cell positions map to one scale degree. The available scales are minor pentatonic, major pentatonic, and Dorian; scale degrees continue upward through octaves from the chosen MIDI root.

Duplicate pitches are removed. At most four pitches play per step, with selection rotated through the candidates to avoid always favoring the low register. This mapping is a compositional choice rather than a unique musical interpretation of the automaton.

MATHS

A trigger occurs when $a_i^t = 1$ and $a_i^{t-1} = 0$. Let $d_i = \lfloor i/2 \rfloor$, root r_0 , and scale intervals $s_0, \dots, s_{K-1}$. Then

$$\text{MIDI}(i) = r_0 + s_{d_i \bmod K} + 12 \left\lfloor \frac{d_i}{K} \right\rfloor.$$

Minor pentatonic uses $(0, 3, 5, 7, 10)$, major pentatonic $(0, 2, 4, 7, 9)$, and Dorian $(0, 2, 3, 5, 7, 9, 10)$.

For J unique candidates at step t , the selection uses indices $(t + j) \bmod J$ for $j = 0, \dots, \min(4, J) - 1$.

07.03 · Rendering a playable WAV

EXPLANATION

The renderer synthesizes each selected note with a fundamental and a weaker second harmonic. Short attack and release envelopes soften discontinuities. Notes last one generation step and decay within that step.

The result is mono, 22,050 Hz, 16-bit PCM in a WAV container. The four-voice limit and bounded per-voice gain constrain the mix before final sample encoding.

This is a transparent synthesized instrument. The cellular evolution determines the note events; the oscillator and envelope determine how those events sound.

MATHS

MIDI pitch p maps to frequency

$$f(p) = 440 \, 2^{(p-69)/12}.$$

For local note time τ and normalized progress v within a step, the envelope is

$$e(\tau, v) = \min(1, \tau/0.008) \min(1, (1 - v)\Delta t/0.035) e^{-2.2v}.$$

One voice contributes

$$x_p(\tau) = \frac{0.13 \, e(\tau, v)}{1.25} [\sin(2\pi f(p)\tau) + 0.25 \sin(4\pi f(p)\tau)].$$

The renderer sums voices, clamps to $[-1, 1]$, and encodes signed 16-bit samples. Rounding sample counts introduces small timing quantization.